A simple test runner just finds test files in a specific directory based on their filenames and executes their contents as CoffeeScript. This test runner can also inject `require` dependencies into a file by munging the file source to put additional `require` expressions at the top of the file before they're run. This is a somewhat unsophisticated and brutish approach, but in the absence of a test framework or a more elegant solution, it works. The following listing shows such a test runner.

Listing 10.8. The test runner

```
fs = require 'fs'
coffee = require 'coffee-script'

test = (file) ->
  fs.readFile file, 'utf-8', (err, data) ->
    it = """
      {fact} = require './fact'
      assert = require 'assert'
      #{data}
    """
    coffee.run it, filename: file

spec = (file) ->
  /[^.]*\.spec\.coffee$/.test file

fs.readdir '.', (err, files) ->
  for file in files
    test file if spec file
```

The test file source has some requires inserted at the top, and then the resulting source is run. This saves you from having to put these specific requires at the top of every file.

Run all the test files in the current directory that have a filename ending with .spec.coffee.

Injecting modules into the source code of a test before passing it to the runtime makes you slightly uneasy, but it works and can make writing tests a bit easier. Making it easier to write tests means you'll be more likely to actually write them, keeping the rest of the program cleaner. Although different test frameworks will have other ways to provide these conveniences for you, the reasons behind them are the same.

What does the setup of the program look like after all this is done? The file and directory structure for the tracking application should now be rearranged to look like this: